# 1. Design a Business Card

Build an app that is functionally similar to this example project. Try not to copy the example project, give it your own personal style.

**Objective:** Fulfill the user stories below and get all the tests to pass to complete the lab.

**User Stories:**

1. You should have a `div` with a `class` attribute with a value of `business-card` that will contain all the other elements.
2. Inside the `.business-card` element, there should be an `img` element with a `class` attribute with a value of `profile-image`. You can set the image source to `https://cdn.freecodecamp.org/curriculum/labs/flower.jpg` if you like. There should be an `alt` with a meaningful description.
3. Inside the `.business-card` element, after the `img` element, you should have three `p` elements with a `class` attribute that has, respectively, a value of `full-name`, `designation`, and `company`.
4. The first `p` element should contain your name.
5. The second `p` element should contain your designation.
6. The third `p` element should have your company name.
7. There should be an `hr` element below the third `p` element.
8. After the `hr` element, there should be two `p` elements. In the first `p` element, you should have an email address as the text. In the second `p` element, the text should be a phone number.
9. After the two `p` elements, there should be an `a` element with a class of `portfolio-link`, the text `Portfolio`, and it should link to a valid URL.
10. There should be an `hr` element after the `a` element containing the portfolio link.
11. You should have another `div` element with a `class` attribute with a value of `social-media`. Within this element, there should be an `h2` element with the text `Connect with me`.
12. Inside the `.social-media` elements, there should be three `a` elements. The `a` elements should have the text `Twitter`, `LinkedIn` and `GitHub` respectively with links to valid websites.
13. You should link the style sheet `styles.css` using the `link` tag in the `head` section of the HTML file.
14. Your page background should be `rosybrown`. The overall font should be `Arial`, with a fallback of `sans-serif`.

15. The `.business-card` selector should have properties to set the element as `300px` wide and a background color of your choice. Also you should set a `padding` of `20px` all around and a top `margin` of `100px`. The text should be center aligned and the font size should be `16px`. The left and right margins should be set to `auto`.
16. The `.profile-image` selector should have a `max-width` property with a value of `100%`.
17. All of the `p` elements should have a top and bottom `margin` of `5px`.
18. All links on the page should not be underlined.

- 1. You should have one main `div` with a `class` attribute with a value of `business-card`.
- 2. Inside the `.business-card` element, there should be an `img` element with a `class` attribute with a value of `profile-image`.
- 3. The image source should be set to a valid image.
- 4. The `alt` attribute of the image should be set to a meaningful description.
- 5. Inside the `.business-card` element, after the `img` element, there should be a `p` element with a `class` attribute with a value of `full-name`.
- 6. After the `.full-name` element, there should be a `p` element with a `class` attribute with a value of `designation`.
- 7. After the `.designation` element, there should be a `p` element with a `class` attribute with a value of `company`.
- 8. The first `p` element, the one with class of `full-name`, should contain your name.
- 9. The second `p` element, the one with a `class` attribute of `designation`, should contain your designation.
- 10. The third `p` element, the one with the `class` attribute with a value of `company`, should contain your company name.
- 11. After the third `p` element, the one with the `class` attribute with a value of `company`, there should be an `hr` element.
- 12. After the first `hr` element, there should be a `p` element with an email address as your text.
- 13. After the email `p` element, there should be another `p` element with a phone number as your text.
- 14. After the phone number `p` element, there should be an `a` element with the class `portfolio-link`.

- 15. The `.portfolio-link` element should have text of `Portfolio`.
- 16. Your `.portfolio-link` element should have a valid `href`.
- 17. After the `a` element containing your portfolio, there should be another `hr` element as a divider.
- 18. After the second `hr` element, there should be a `div` element with a `class` attribute with a value of `social-media`.
- 19. Inside the `.social-media` element, there should be an `h2` element with the text `Connect with me`.
- 20. Inside the `.social-media` element there should be three `a` elements.
- 21. The first `a` element should have the text `Twitter`.
- 22. The `href` of the first `a` element should point to a valid link.
- 23. The second `a` element should have the text `LinkedIn`.
- 24. The `href` of the second `a` element should point to a valid link.
- 25. The third `a` element should have the text `GitHub`.
- 26. The `href` of the third `a` element should point to a valid link.
- 27. Your `link` element should be within your `head` element.
- 28. Your `link` element should have a `rel` attribute with the value `stylesheet`.
- 29. Your `link` element should have an `href` attribute with the value `styles.css`.
- 30. You should set the page background color in the `body` selector to `rosybrown`.
- 31. You should set the page font to `Arial` in the `body` element with a fallback of `sans-serif`.
- 32. Your `.business-card` selector should have a `width` property with a value of `300px`.
- 33. Your `.business-card` selector should have a `background-color` property with a valid color value.
- 34. Your `.business-card` selector should have a `padding` property with a value of `20px`.
- 35. Your `.business-card` selector should have a top margin of `100px`.
- 36. The text inside the `.business-card` element should be center-aligned.
- 37. The font size of the text inside the `.business-card` element should be `16px`.
- 38. The left and right margins of the `.business-card` element should be set to `auto`.
- 39. Your `.profile-image` selector should have a `max-width` property with a value of `100%`.
- 40. Your `p` elements should have a top and bottom `margin` of `5px`.
- 41. The links of the page should have no underline.

# 2. Build a Stylized To-Do list

Build an app that is functionally similar to this example project. Try not to copy the example project, give it your own personal style.

In this lab, you will practice the different styles that can be applied to links when they are hovered over, focused, clicked, and visited.

**Objective:** Fulfill the user stories below and get all the tests to pass to complete the lab.

**User Stories:**

1. You should have one unordered list with the class `todo-list`.
2. Inside the unordered list, you should have four list items.
3. Inside each list item, there should be:

- An `input` element with the type `checkbox` and `id` set to a unique value.
- A `label` element with the `for` attribute set to the corresponding `input` element's `id`.
- An unordered list with the class `sub-item`.
- A list item with an anchor element in it. The anchor should have the class `sub-item-link`, a valid `href` value, and a `target` value that makes the link open in a new tab.

4. Your `a` elements should not have any text decorations.
5. You should set the text color of unvisited links to a color of your choice.
6. When your links are visited, the color should change to another color of your choice.
7. When your links are hovered over, the color should change to another color of your choice.
8. When your links are focused, there should be a colored outline around the link.
9. When your links are clicked, the color should change to another color of your choice.

**Note:** Be sure to link your stylesheet in your HTML and apply your CSS.

- 1. You should have one unordered list with the class `todo-list`.
- 2. You should have four list items inside the unordered list.
- 3. The `li` inside the `ul` with the class `todo-list` should contain an `input` element with the `type` of `checkbox`.
- 4. The `li` inside the `ul` with the class `todo-list` should contain a `label` element.

- 5. All `input` elements should have an `id`.
- 6. All `label` elements should have a `for` attribute.
- 7. All `label` elements should have some text.
- 8. The `id` and `for` attributes of the `input` and `label` elements pairs, should have corresponding values.
- 9. After the `label` elements, there should be an unordered list with the class `sub-item`.
- 10. The `li` inside the `ul` with the class `sub-item` should have an anchor element with the class `sub-item-link`.
- 11. All anchor elements should have a valid `href` attribute.
- 12. All anchor elements should have text.
- 13. Each `a` element should have a `target` attribute with the value of `_blank`.
- 14. Your `a` elements should not have any text decorations.
- 15. Unvisited links should have a text color set to a color of your choice.
- 16. The links should change color when hovered over.
- 17. The links should change color when they are being clicked.
- 18. The links should have an outline when focused.
- 19. The links should change color once visited.

# 3. Build a Job Application Form

Build an app that is functionally similar to this example project. Try not to copy the example project, give it your own personal style.

**Objective:** Fulfill the user stories below and get all the tests to pass to complete the lab.

**User Stories:**

1. You should have a `div` element with the class `container`.
2. Inside the `div` element, you should have a `form` element.
3. The form should contain an `input` element with the type `text` and the id `name` for entering the user's full name.
4. You should have another `input` element with the type `email` and the id `email` for entering the user's email address.
5. The form should include a `select` element with the id `position` that allows users to select a job position.
6. You should have a `fieldset` element with class of `radio-group`.
7. Inside `.radio-group` you should have a set of `input` elements with the type `radio` and relevant labels for selecting availability options (e.g., Full-Time, Part-Time). The group `name` should be `availability`.

8. You should have a `textarea` element with the id `message` for entering a message.
9. You should associate every `input` element with a `label` element.
10. You should have a `button` element with the type `submit` for submitting the form.
11. Add a `:focus` pseudo-class to the `input` and `textarea` elements to change their border color when focused.
12. The `input`, `select` and `textarea` elements should have an `:invalid` pseudo-class that changes the border color to red when invalid input is detected.
13. The `input`, `select` and `textarea` elements should have a `:valid` pseudo-class that changes the border color to green when valid input is entered.
14. The `button` element should have a `:hover` pseudo-class that changes the background color when hovered over.
15. Use the `:checked` pseudo-class on `.radio-group` `input[type="radio"]` to add a border color, background color and a box shadow when the radio button is selected.
16. Use the `:checked` pseudo-class on radio buttons to change the text color of the associated `label` when the option is selected.
17. Add a `:first-of-type` pseudo-class to the `input` element to style the first input field differently. (e.g., rounded corners).

**Note:** Be sure to link your stylesheet in your HTML and apply your CSS.

- 1. You should have a `div` element with the class `container`.
- 2. Inside the `div` element, you should have a `form` element.
- 3. The form should contain an `input` element with the type `text` and the id `name` for entering the user's full name.
- 4. You should have another `input` element with the type `email` and the id `email` for entering the user's email address.
- 5. The form should include a `select` element with the id `position` with some `option` elements.
- 6. You should have a `fieldset` or `section` element with the class `.radio-group`.
- 7. Inside `.radio-group` you should have a group of `input` elements with the type `radio` for selecting availability options. The group `name` should be `availability`.
- 8. You should have a `textarea` element with the id `message` for entering a message.

- 9. You should associate every `input` element with a `label` element.
- 10. You should have a `button` element with the type `submit` for submitting the form.
- 11. You should add a `:focus` pseudo-class to the `input` and `textarea` elements to change their border color when focused. Use a list selector in the given order.
- 12. The `input`, `select` and `textarea` elements should have an `:invalid` pseudo-class that changes the border color to red when invalid input is detected. Use a list selector in the given order.
- 13. The `input`, `select` and `textarea` elements should have a `:valid` pseudo-class that changes the border color to green when valid input is entered. Use a list selector in the given order.
- 14. The `button` element should have a `:hover` pseudo-class that changes the background color when hovered over.
- 15. You should use the `:checked` pseudo-class on `.radio-group` `input[type="radio"]` to add a border color when the radio button is selected.
- 16. You should use the `:checked` pseudo-class on `.radio-group` `input[type="radio"]` to add a background color when the radio button is selected.
- 17. You should use the `:checked` pseudo-class on `.radio-group` `input[type="radio"]` to add a box shadow when the radio button is selected.
- 18. You should use the `:checked` pseudo-class on radio buttons to change the text color of the associated `label` when the option is selected.
- 19. Add a `:first-of-type` pseudo-class to the `input` elements to style the first input field differently.

# 4. Design a Set of Colored Boxes

Build an app that is functionally similar to this example project. Try not to copy the example project, give it your own personal style.

In this lab, you'll practice using CSS colors by designing boxes.

**Objective:** Fulfill the user stories below and get all the tests to pass to complete the lab.

1. You should set the background color for `body` to `#f4f4f4`.

2. You should have a `div` with a class of `color-grid` to hold all your color elements.
3. You should have five `div` elements within the `.color-grid` element.
4. The five `div` elements should each have a class of `color-box` and `color#`, where `#` is the number of the order of that `div`. For example: `color1` for the first `div`, `color2` for the second, and so on.
5. The `.color-box` class should have a set `width` and `height` so your `div` elements are visible on the page.
6. The `.color1` element should have a `background-color` that uses hexadecimal color value.
7. The `.color2` element should have a `background-color` that uses an RGB color value.
8. The `.color3` element should have a `background-color` that uses a predefined (word) color value.
9. The `.color4` element should have a `background-color` that uses a HSL color value.
10. The `.color5` element should have a `background-color` set.

**Note:** Be sure to link your stylesheet in your HTML and apply your CSS.

- 1. `body` should have a background color of `#f4f4f4`.
- 2. You should have a `div` element with a class of `color-grid`.
- 3. You should have five `div` elements within the `.color-grid` element.
- 4. Each of the five `div` elements should each have a class of `color-box` and `color#`—substitute the order of the `div` for the `#` symbol.
- 5. The `.color-box` element should have a set `width` and `height`.
- 6. The `.color1` element should have a hexadecimal background color.
- 7. The `.color2` element should have an RGB background color.
- 8. The `.color3` element should have a predefined (word) background color.
- 9. The `.color4` element should have a HSL background color.
- 10. The `.color5` element should have a background color set.

# 5. Design a Feature Selection Page

Build an app that is functionally similar to this example project. Try not to copy the example project, give it your own personal style.

**Objective:** Fulfill the user stories below and get all the tests to pass to complete the lab.

**User Stories:**

1. Your page should have an `h1` element with the text `Feature Selection`.
2. Your page should have a `div` element with the class `feature-card-container`.
3. Your page should have at least two `label` elements each with the class `feature-card` inside the `div`.
4. Each of your `label` elements should have label text and an `input` element of type `checkbox` in them.
5. Create a selector that targets the checkboxes, and apply the following styling:

- All of your checkbox elements should be set to `appearance: none;` in your CSS.
- All of your checkbox elements should have a border width, color and style of your choosing.

6. When the checkbox is checked, it should display a checkmark `✓` in the center of the checkbox.
7. When the checkbox is checked, the background color of the checkbox should change to a color of your choosing.
8. When the checkbox is checked, the border color of the checkbox should change to a color of your choosing.

Steps:

- 1. You should have an `h1` element.
- 2. Your `h1` should have a text of `Feature Selection`.
- 3. You should have a `div` element with the class `feature-card-container`.
- 4. You should have at least two `label` elements inside `.feature-card-container`.
- 5. Each `label` element inside `.feature-card-container` should have the class `feature-card`.
- 6. Inside each `label` element you should have label text.
- 7. Inside each label element there should be one `input` element of type `checkbox`.
- 8. You should have a selector for the checkboxes.
- 9. Your checkboxes should be set to `appearance: none`.
- 10. Your checkboxes should have a border width of your choosing.
- 11. Your checkboxes should have a border color of your choosing.
- 12. Your checkboxes should have a border style of your choosing.

- 13. When the checkboxes are selected, there should be a checkmark ✓ in the center of the checkbox.
- 14. When the checkbox is checked, the background color of the checkbox should change to a color of your choosing.
- 15. When the checkbox is checked, the border color of the checkbox should change to a color of your choosing.

# 6. Build a Confidential Email Page

Build an app that is functionally similar to this example project. Try not to copy the example project, give it your own personal style.

**Objective:** Fulfill the user stories below and get all the tests to pass to complete the lab.

**User Stories:**

1. You should have a `main` element with an `id` of `email`.
2. Your `#email` element should have `padding` of `50px`, a top margin of `50px`, a `width` of `500px`, and a `border` that's `2px`.
3. The total width of your `#email` element, including paddings and borders, should be `500px`.
4. You should have two `div` elements, one with an id of `confidential` and the other with an id of `top-secret`, within your `#email` element.
5. Your `#confidential` and `#top-secret` elements should have a `display` of `inline-block`.
6. Your `#confidential` and `#top-secret` elements should have a `padding`, a left margin, and a `border`.
7. The `#confidential` element should have the text `CONFIDENTIAL`.
8. The `#top-secret` element should have the text `TOP SECRET`.
9. Your `#confidential` and `#top-secret` elements should be rotated using a CSS transform.
10. You should have at least three paragraph elements within your `#email` element.
11. You should have at least three `span` elements with a class of `blurred`, within your paragraph elements.
12. You should have a `blurred` selector that blurs the element `3px` using a CSS filter.

- 1. You should have a `main` element with an `id` of `email`.
- 2. You should have an `#email` selector that sets its elements' `padding` on all sides to `50px`.
- 3. You should have an `#email` selector that sets its elements' `margin-top` to `50px`.
- 4. You should have an `#email` selector that sets its elements' `width` to `500px`.
- 5. Your `#email` element should have a `2px` border.

- 6. Your `#email` element should have a `box-sizing` of `border-box`.
- 7. You should have a `div` element with an `id` of `confidential` within your `#email` element.
- 8. You should have a `div` element with an `id` of `top-secret` within your `#email` element.
- 9. Your `#confidential` element should have its `display` set to `inline-block`.
- 10. Your `#top-secret` element should have its `display` set to `inline-block`.
- 11. Your `#confidential` element should have a `padding` on all sides.
- 12. Your `#top-secret` element should have a `padding` on all sides.
- 13. Your `#confidential` element should have a left margin.
- 14. Your `#top-secret` element should have a left margin.
- 15. Your `#confidential` element should have a `border`.
- 16. Your `#top-secret` element should have a `border`.
- 17. Your `#confidential` element's text should be `CONFIDENTIAL`. You have either omitted the text, or have a typo.
- 18. Your `#top-secret` element's text should be `TOP SECRET`. You have either omitted the text, or have a typo.
- 19. Your `#confidential` element should use a CSS `transform` to rotate the element.
- 20. Your `#top-secret` element should use a CSS `transform` to rotate the element.
- 21. You should have at least three paragraph elements within your `#email` element.
- 22. You should have at least three `span` elements with a class of `blurred` within your paragraphs.
- 23. Your `.blurred` elements should not be empty.
- 24. You should have a `.blurred` selector that set its elements `filter` to `blur(3px)`.

# 7. Design a Pricing Plans Layout Page

Build an app that is functionally similar to this example project. Try not to copy the example project, give it your own personal style.

**Objective:** Fulfill the user stories below and get all the tests to pass to complete the lab.

**User Stories**

1. Your page should have an `h1` element with the text `Pricing Plans`.

2. Your page should have a `div` element with the class `pricing-container` below the `h1` element.
- The `.pricing-container` selector should have a `display` property with the value of `flex` and a `flex-wrap` property with the value of `wrap`.
3. Within the `.pricing-container` element, you should have three `div` elements with the class `pricing-card` to represent the pricing plans.
- One of the `.pricing-card` elements should have the class `basic-plan` in addition to the `pricing-card` class.
- One of the `.pricing-card` elements should have the class `pro-plan` in addition to the `pricing-card` class.
- One of the `.pricing-card` elements should have the class `premium-plan` in addition to the `pricing-card` class.
4. Your `.basic-plan` element should have an `h2` element with the text `Basic`.
5. Your `.basic-plan` element should have a `p` element with the text `$9/month`.
6. Your `.pro-plan` element should have an `h2` element with the text `Pro`.
7. Your `.pro-plan` element should have a `p` element with the text `$19/month`.
8. Your `.premium-plan` element should have an `h2` element with the text `Premium`.
9. Your `.premium-plan` element should have a `p` element with the text `$29/month`.
10. Each of your `.pricing-card` elements should:
- Use Flexbox to stack its content in a column and justify the space between the children using `space-between`.
- Set the `flex` property to `0 0 200px` to give it a consistent width and prevent it from growing or shrinking in the layout.
- Set the `border` property to `2px solid black` to see how the different cards take up space.
11. The `.basic-plan` element should appear first in the layout. You should use the `order` property for this.
12. The `.pro-plan` element should appear second in the layout. You should use the `order` property and set its `flex-grow` property to `2` so it takes up more space than the other plans.
13. The `.premium-plan` element should come last in the layout. You should use the `order` property for this.

Steps:

- 1. You should have an `h1` element.
- 2. Your page should have an `h1` element with the text `Pricing Plans`.
- 3. You should have a `div` element below the `h1` element.
- 4. Your `div` element should have the class `pricing-container`.
- 5. You should have a `.pricing-container` selector.
- 6. Your `pricing-container` selector should have the property `display` with a value of `flex`.
- 7. Your `.pricing-container` selector should have the property `flex-wrap` with a value of `wrap`.
- 8. You should have three `div` elements with the class `pricing-card` within the `.pricing-container` element.
- 9. One of your `.pricing-card` elements should have the class `basic-plan`.
- 10. One of your `.pricing-card` elements should have the class `premium-plan`.
- 11. One of your `.pricing-card` elements should have the class `pro-plan`.
- 12. You should have a `.pricing-card` selector.
- 13. Your `.pricing-card` selector should have the property `display` with a value of `flex`.
- 14. Your `.pricing-card` selector should have the property `flex-direction` with a value of `column`.
- 15. Your `.pricing-card` selector should have the property `justify-content` with a value of `space-between`.
- 16. Your `.pricing-card` selector should have the property `flex` with a value of `0 0 200px`.
- 17. Your `.pricing-card` selector should have the property `border` with a value of `2px solid black`.
- 18. Your `.basic-plan` selector should have the property `order` with a value of `0`.
- 19. Your `.basic-plan` element should have an `h2` element with the text `Basic`.
- 20. Your `.basic-plan` element should have a `p` element with the text `$9/month`.
- 21. Your `.pro-plan` element should have an `h2` element with the text `Pro`.
- 22. Your `.pro-plan` element should have a `p` element with the text `$19/month`.

- 23. Your `.pro-plan` element should have the property `order` with a value of `1`.
- 24. Your `.pro-plan` element should have the property `flex-grow` with a value of `2`.
- 25. Your `.premium-plan` element should have an `h2` element with the text `Premium`.
- 26. Your `.premium-plan` element should have a `p` element with the text `$29/month`.
- 27. Your `premium-plan` element should be the last element in the layout and have the property `order` and the value `2`.

# 8. Build a Personal Portfolio

Build an app that is functionally similar to this example project. Try not to copy the example project, give it your own personal style.

**Objective:** Fulfill the user stories below and get all the tests to pass to complete the lab.

**User Stories:**

1. Your portfolio should have a welcome section with an `id` of `welcome-section`.
2. The welcome section should have an `h1` element that contains text.
3. Your portfolio should have a projects section with an `id` of `project-section`.
4. The projects section should contain at least one element with a `class` of `project-tile` to hold a project.
5. The projects section should contain at least one link to a project.
6. Your portfolio should have a navbar with an id of `navbar`.
7. The navbar should contain at least one link that you can click on to navigate to different sections of the page.
8. Your portfolio should have a link with an id of `profile-link`, which opens your GitHub or freeCodeCamp profile in a new tab.
9. Your portfolio should have at least one media query.
10. The height of the welcome section should be equal to the height of the viewport.
11. The navbar should always be at the top of the viewport.

Steps:

- 1. Your portfolio should have a "Welcome" section with an `id` of `welcome-section`.
- 2. Your `#welcome-section` element should contain an `h1` element.
- 3. You should not have any empty `h1` elements within `#welcome-section` element.
- 4. You should have a "Projects" section with an `id` of `project-section`.
- 5. Your portfolio should contain at least one element with a class of `project-tile`.
- 6. Your `#project-section` element should contain at least one `a` element.
- 7. Your portfolio should have a navbar with an `id` of `navbar`.
- 8. Your `#navbar` element should contain at least one `a` element whose `href` attribute starts with `#`.
- 9. Your portfolio should have an `a` element with an `id` of `profile-link`.
- 10. Your `#profile-link` element should have a `target` attribute of `_blank`.
- 11. Your portfolio should use at least one media query.
- 12. Your `#navbar` element should always be at the top of the viewport.